

TECH CHALLENGE — FASE 3

AUTOMAÇÃO DE INFRAESTRUTURA E CICLO DE VIDA COM IaC, CI/CD, DEVSECOPS E GITOPS — TOGGLEMASTER

GABRIEL PINELLI SILVA
JOÃO VITOR DE JESUS CIARDULLO
DOUGLAS DEVEZA DOS SANTOS
JOÃO CARLOS DA SILVA BRITO
JOÃO GABRIEL DA CRUZ SALES
Cuiabá — MT
2026

Relatório de Entrega: Tech Challenge — Fase 3

Curso: Pós-Graduação em DevOps & Arquitetura Cloud (FIAP)

Projeto: Automação de Infraestrutura e Ciclo de Vida — ToggleMaster

Grupo: 203

1. Identificação do Grupo

Integrante	RM	Username Discord
Gabriel Pinelli Silva	RM373763	tocacelly
João Vitor de Jesus Ciardullo	RM372155	joaozinho1403
Douglas Deveza dos Santos	RM373827	d0guera
João Carlos da Silva Brito	RM371738	durmiand
João Gabriel da Cruz Sales	RM372444	jgabrieldev

2. Links Obrigatórios de Entrega

- **Repositório (Git):** <https://github.com/fiap-devops-arqcloud-2026/tech-challenge-03>
- **Vídeo de Demonstração (YouTube):** [PREENCHER APÓS A GRAVAÇÃO]
- **Documentação técnica:** [README.md](#) no repositório, com arquitetura, decisões e instruções de reprodução

3. O Problema e o Que Foi Automatizado

A Fase 2 entregou os cinco microsserviços do ToggleMaster rodando em Kubernetes na AWS — mas **toda a infraestrutura foi criada à mão no console**, e o deploy dependia de `kubectl apply` executado da máquina de cada desenvolvedor. O enunciado da Fase 3 descreve exatamente as consequências disso: conflitos de versão entre quem aplica, e um ambiente de homologação que leva dias para ser recriado.

A ordem desta fase é literal: **"Se não está no código, não existe"**. O trabalho consistiu em substituir cada passo manual por código versionado:

O que era manual na Fase 2	Como ficou na Fase 3
VPC, subnets, EKS, RDS, Redis, SQS, DynamoDB e ECR criados no console	Terraform em três camadas, com estado remoto em S3
Imagens construídas e enviadas a partir da máquina do desenvolvedor	Pipeline no GitHub Actions, com build, linter, SAST, SCA e scan de imagem
Credenciais estáticas da AWS guardadas em arquivo	Federação OIDC para o CI e IRSA para os pods — nenhuma chave estática no projeto
<code>kubectl apply</code> da máquina local	ArgoCD observando o Git e reconciliando o cluster sozinho

4. Infraestrutura como Código (Terraform)

Todo o provisionamento é declarativo, organizado em **módulos próprios** e dividido em **três camadas com estado independente** no mesmo bucket S3:

Camada	O que provisiona	Ciclo de vida	Custo parado
<code>terraform/</code> (base)	VPC, subnets, IGW, route tables, 5 repositórios ECR, fila SQS + DLQ, tabela DynamoDB, provedor OIDC e role do CI	permanente	~US\$ 0,00
<code>terraform/</code> <code>cluster/</code>	cluster EKS, node group, 2 instâncias RDS PostgreSQL, ElastiCache Redis, roles de IRSA	sobe e desce a cada sessão	~US\$ 0,37/h
<code>terraform/</code> <code>k8s/</code>	namespace, os 5 Secrets, StorageClass <code>gp3</code> , ArgoCD e a Application	acompanha o cluster	~US\$ 0,00

Por que separar em camadas. Com estado único, o `terraform destroy` feito para parar de gastar crédito levaria junto os repositórios ECR — e, com eles, as imagens já publicadas. O CI teria de reconstruir e reenviar as cinco imagens antes de cada sessão de trabalho. Com estados separados, destrói-se apenas o que cobra por hora.

Estado remoto. O backend é um bucket S3 com criptografia, versionamento e bloqueio de acesso público, usando a flag `use_lockfile` para o *lock* nativo (exigência opcional citada no enunciado). Um arquivo de estado por camada: `prod/base.tfstate`, `prod/cluster.tfstate` e `prod/k8s.tfstate`. Isso não é formalidade: o `terraform.tfstate` armazena as senhas dos bancos **em texto puro** — mantê-lo local, ou pior, versionado, seria um vazamento.

Módulos. Sete módulos próprios (`ecr`, `eks`, `elasticache`, `iam-ci`, `irsa`, `messaging`, `rds`) mais o módulo oficial de VPC da comunidade. A camada base foi aplicada na AWS em 07/09/2026, criando 33 recursos; a camada do cluster planeja 35 recursos adicionais.

5. Pipeline de CI e DevSecOps

São **nove workflows** no GitHub Actions: dois reutilizáveis (um para Go, um para Python), cinco chamadores — um por microsserviço —, um de validação do Terraform e um de teste integrado dos cinco serviços via Docker Compose.

Cada pipeline dispara em **Pull Request e em push na `main`**, com filtro de caminho para que alterar um serviço não acione os outros. Os estágios:

Job	Ferramenta	Papel
Build e testes	<code>go build / compileall + pytest</code>	compila e roda testes, quando existirem
Linter	<code>golangci-lint</code> (Go), <code>flake8</code> e <code>pylint</code> (Python)	análise estática de estilo e defeito
SAST	<code>gosec</code> (Go), <code>bandit</code> (Python)	vulnerabilidade no código fonte
SCA	Trivy em modo <code>fs</code>	vulnerabilidade nas dependências
Imagem	Docker build + Trivy <code>image</code> + push no ECR	constrói, escaneia e publica
GitOps	<code>kustomize edit set image</code>	grava a tag nova na área de GitOps

A regra de bloqueio, implementada ao pé da letra. O job de imagem declara `needs: [build, lint, sast, sca]`. Se qualquer um dos quatro falhar, ele **não roda** — aparece como *skipped*, não como *failed*. O enunciado pede que uma vulnerabilidade crítica faça o pipeline "falhar e não prosseguir", e a diferença entre falhar e não prosseguir é exatamente essa: a imagem vulnerável nunca chega a ser construída, muito menos publicada.

Autenticação sem chave estática. O pipeline assume uma role da AWS por federação OIDC, com a policy de confiança restrita a este repositório. Não há `AWS_ACCESS_KEY_ID` nem `AWS_SECRET_ACCESS_KEY` guardados nos Secrets do GitHub.

Tag da imagem. Cada imagem vai para o ECR com o hash do commit, no padrão `v1.0.0-a1b2c3d` indicado no enunciado — nunca apenas `latest`.

6. Entrega Contínua e GitOps

A área de GitOps é a pasta `gitops/` no monorepo — alternativa que o enunciado aceita explicitamente —, escrita em **Kustomize**, com `base/` por serviço e um único overlay `prod/`.

O ciclo completo, sem intervenção humana depois do merge:

1. o merge na `main` dispara os pipelines dos serviços alterados;
2. passando por build, linter, SAST e SCA, a imagem é construída, escaneada e publicada no ECR;
3. o último job roda `kustomize edit set image` e **comita a tag nova** em `gitops/overlays/prod/kustomization.yaml`, com `[skip ci]` para não gerar um laço infinito;

4. o ArgoCD, que observa essa pasta na branch `main`, detecta o commit, renderiza o Kustomize e aplica a diferença no cluster.

Não há um único `kubectl apply` nos workflows. O pipeline escreve no Git; o ArgoCD é quem toca o cluster. Uma Application gerencia os cinco microsserviços, com sincronização automática e *self-heal* — alterações feitas diretamente no cluster são desfeitas, que é a resposta direta ao problema descrito no enunciado.

O ArgoCD é instalado por Terraform, usando o provider `helm` com a versão do chart fixada.

7. Desafios Encontrados e Decisões Tomadas

7.1 O plano gratuito da AWS recusou a terceira instância RDS

O enunciado pede três instâncias RDS PostgreSQL. A conta, no plano gratuito novo da AWS, **recusa a terceira** com a mensagem `maximum number of instances available with free plan accounts`.

Decisão: duas instâncias RDS (`auth_db` e `flags_db`) e o terceiro banco, `targeting_db`, como StatefulSet PostgreSQL dentro do cluster, com disco EBS persistente. É o mesmo arranjo usado na Fase 2 e validado com o professor.

7.2 O mesmo plano recusou o tipo de máquina planejado

O `t3.medium` previsto para os nós foi recusado como não elegível ao Free Tier. **Decisão:** `c7i-flex.large` (2 vCPU / 4 GB), equivalente e permitida.

7.3 A versão do Kubernetes custava seis vezes mais

O planejamento inicial apontava EKS 1.31. Consulta à API da AWS mostrou que essa versão já havia saído do suporte padrão, migrando para suporte estendido — o que leva o control plane de **US\$ 0,10/h para US\$ 0,60/h**, seis vezes mais, sem nenhum ganho para o projeto.

Decisão: EKS **1.34**, em suporte padrão até dezembro de 2026.

7.4 O pipeline verde escondia uma vulnerabilidade crítica real

Ao implementar a regra de bloqueio, o Trivy acusou o **CVE-2026-56854**, classificado como crítico, na biblioteca `golang.org/x/crypto` v0.20.0 usada pelos serviços em Go. A vulnerabilidade **já estava no projeto**, e os pipelines anteriores passavam verdes porque os passos de scan usavam `continue-on-error` — eles imprimiam o achado e devolviam sucesso.

Decisão: atualizar para a v0.55.0. A correção teve efeito cascata: a biblioteca nova exigiu Go 1.25, o que obrigou a atualizar os Dockerfiles e a versão usada pelo SAST, e a migrar o `golangci-lint` da linha 1.x para a 2.x — que, por sua vez, apontou três tratamentos de erro ausentes, também corrigidos.

O aprendizado, que vale registrar: um pipeline verde não prova que o código é seguro. Prova que ninguém configurou o pipeline para reclamar.

7.5 Vulnerabilidades críticas sem correção publicada

Com a regra de bloqueio no rigor literal do enunciado, o scan da imagem passou a acusar **três CVEs críticos no pacote** `perl-base` da imagem base `python:3.12-slim`, usada pelos três serviços em Python. Nenhum deles tem versão corrigida publicada pelo Debian, e `perl-base` é um pacote essencial — não pode ser removido sem quebrar a imagem.

Decisão: manter a regra estrita (`ignore-unfixed: false`) e registrar os três CVEs **nominalmente** em um arquivo `.trivyignore`, cada um com justificativa escrita e data de revisão. A alternativa comum — desligar a flag de vulnerabilidades sem correção — esconderia uma classe inteira de achados, em silêncio e para sempre, inclusive os que surgissem depois. Com a lista nominal, qualquer crítico fora dela continua derrubando o pipeline, e cada exceção aparece no diff de qualquer Pull Request.

Os três serviços em Python são os únicos afetados: a imagem `alpine:3.20` usada pelos serviços em Go foi medida com **zero** vulnerabilidades críticas.

7.6 O Terraform não conseguia instalar o ArgoCD de primeira

O recurso que cria a Application do ArgoCD valida o tipo dela contra o cluster ainda na fase de *plan* — mas esse tipo só passa a existir depois que o próprio ArgoCD é instalado, o que acontece na fase de *apply*. Em um cluster novo, o apply direto falha com `no matches for kind "Application"`.

Decisão: documentar o primeiro apply da camada em **duas etapas** — uma limitada à instalação do ArgoCD, outra para o restante. Foram avaliadas e descartadas duas alternativas: adicionar um provider de terceiros ao projeto, e criar a Application fora do Terraform, o que tiraria do código exatamente o objeto que demonstra o GitOps.

7.7 Sem Ingress nem Load Balancer

A Fase 2 usava um Ingress Nginx com cinco rotas. O enunciado da Fase 3 não exige exposição externa, e um Application Load Balancer custaria entre US\$ 16 e US\$ 20 por mês ligado continuamente.

Decisão: remover o Ingress. O acesso durante a demonstração é por `kubect1 port-forward`.

7.8 Disciplina de branches

Durante o desenvolvimento houve pushes diretos na `main`, e a branch `dev` chegou a ficar dezenove commits atrás. **Decisão:** todo trabalho humano parte da `dev` e chega à `main` por Pull Request. A única exceção é o commit automático de tag feito pelo pipeline, que vai direto para a `main` de propósito — é dela que o ArgoCD lê o estado desejado.

8. Estimativa de Custos da AWS

8.1 Estimativa oficial — AWS Pricing Calculator

Consulta feita em **11/09/2026**, região **Leste dos EUA (Ohio)**, `us-east-2`, considerando o ambiente completo ligado durante o mês inteiro (730 horas).

Link público da estimativa: <https://calculator.aws/#/estimate?id=1717508852ab38c3aefc4acf0aa7f3de4a797ff9>

Serviço	Configuração estimada	Mensal (US\$)
Amazon EKS	1 cluster Kubernetes 1.34 em suporte padrão	73,00
Amazon EC2	2 × <code>c7i-flex.large</code> , Linux, sob demanda, 20 GB por nó	126,99
Amazon RDS for PostgreSQL	2 × <code>db.t3.micro</code> , Single-AZ, 20 GB gp3 por instância	30,88
Amazon ElastiCache	1 × Redis <code>cache.t3.micro</code>	12,41
Amazon VPC	1 NAT Gateway, 1 IPv4 público e 1 GB processado	36,54
Amazon EBS	1 volume gp3 de 5 GB, 3.000 IOPS e 125 MB/s	0,40
AWS Secrets Manager	2 segredos por 30 dias e 1.000 chamadas de API	0,81
Total mensal	730 horas	281,03
Total em 12 meses	sem desconto	3.372,36

[aws](#) pricing calculator

Feedback
 idioma: Português ▼
 Entre em contato com a equipe de vendas ▶
 Criar uma conta da AWS

Your ToggleMaster - FIAP Fase 3 was last updated on 11 de setembro de 2026.

This is a read-only document. Your estimate below reflects pricing as on 11 de setembro de 2026. To view current pricing, select the Update estimate button. The estimate will need to be saved via share or export after changes are made.

[Atualizar estimativa](#)

AWS Pricing Calculator > ToggleMaster - FIAP Fase 3

ToggleMaster - FIAP Fase 3

Data da estimativa: 11 de setembro de 2026

Resumo da estimativa [Informações](#)

Custo inicial 0,00 USD	Custo mensal 281,03 USD	Custo total de 12 months 3.372,36 USD Inclui um custo inicial
---------------------------	----------------------------	--

Conceitos básicos da AWS

Comece a usar gratuitamente

Entre em contato com a equipe de vendas

ToggleMaster - FIAP Fase 3

 Localizar recursos

☐	Nome do serviço ▼	Custo inicial ▼	Custo mensal ▼	Descrição ▼	Região ▼	Resumo da configuração ▼
☐	Amazon EKS	0,00 USD	71,00 USD	EKS ToggleMaster - 1 cluster K...	Leste dos EUA (Ohio)	Número de clusters do EKS (1). Número de nós híbridos (1 por mba)
☐	Amazon EC2	0,00 USD	126,99 USD	Nox EKS - 2 instâncias c7i-flex...	Leste dos EUA (Ohio)	Locação (instâncias compartilhadas), Sistema operacional (Linux), Carga de trabalho (Consistent, ...
☐	Amazon RDS for PostgreSQL	0,00 USD	30,88 USD	RDS PostgreSQL ToggleMaster...	Leste dos EUA (Ohio)	Quantidade de armazenamento (20 GiB), Volume de armazenamento (SSD de uso geral (gp3)), No...
☐	Amazon ElastiCache	0,00 USD	12,41 USD	ElastiCache Redis ToggleMaste...	Leste dos EUA (Ohio)	Mecanismo (Redis OSS), Nó (R), Tipo de instância (cache.r6g.12xlarge), Utilização (somente sob ...
☐	Amazon Virtual Private Cloud (...)	0,00 USD	36,54 USD	VPC ToggleMaster - 1 NAT Gat...	Leste dos EUA (Ohio)	Número de endereços IPv4 públicos em uso (1), Número de endereços IPv4 públicos inativos (0) N...
☐	Amazon Elastic Block Store (EBS)	0,00 USD	0,40 USD	EBS gp3 ToggleMaster - volum...	Leste dos EUA (Ohio)	Número de volumes (1), Duração média de volume (730 horas por mba), Quantidade de armazen...
☐	AWS Secrets Manager	0,00 USD	0,81 USD	Secrets Manager ToggleMaster...	Leste dos EUA (Ohio)	Número de segredos (2), Duração média de cada segredo (10 dias), Número de chamadas de API (...)

Confirmação

A Calculadora de Preços da AWS fornece apenas uma estimativa de suas taxas da AWS e não inclui nenhuma taxa aplicável. Suas taxas reais dependem de vários fatores, inclusive de seu uso real dos serviços da AWS. [Saiba mais](#) ▶

Privacidade | Termos do site | Preferências de cookies | © 2026, Amazon Web Services, Inc. ou sua afiliada. Todos os direitos reservados.

8.2 O que esse número significa — e por que o ambiente é efêmero

Os **US\$ 281,03/mês** acima correspondem ao ambiente ligado 24 horas por dia, todos os dias. Não é assim que o projeto opera.

A camada cara — EKS, node group, RDS e ElastiCache — **só precisa existir durante as sessões de ensaio e de gravação**, e é destruída ao fim de cada uma. Na prática:

Cenário	Custo
Ambiente ligado o mês inteiro	~US\$ 281,03
Custo por hora com tudo no ar	~US\$ 0,385
Uma sessão de trabalho de 3 horas	~US\$ 1,15
Camada base parada entre sessões (VPC, ECR, SQS, DynamoDB, S3)	~US\$ 0,00

Foi essa diferença — de quase trezentos dólares por mês para pouco mais de um dólar por sessão — que motivou a divisão do Terraform em camadas com estado independente descrita na seção 4. Sem ela, destruir o ambiente para conter custo levaria junto os repositórios ECR e as imagens já publicadas.

8.3 Limites desta estimativa

- S3, ECR, SQS, DynamoDB e transferência de dados variam com o uso e não entram no total acima. O volume de um projeto acadêmico tende a ser pequeno, mas o custo real só aparece na fatura.
- Preços, impostos e câmbio podem mudar depois de 11/09/2026.
- A estimativa considera **duas** instâncias RDS e o terceiro banco em pod, conforme a exceção registrada na seção 7.1.
- O NAT Gateway responde por US\$ 36,54 do total mensal e **não é destruído** junto com o cluster: ele vive na camada base. Desligá-lo ao fim de cada sessão é um passo próprio do procedimento de encerramento.

Todos os recursos recebem as tags `project = fiap` e `phase = 3` automaticamente, via `default_tags` no provider. O AWS Cost Explorer pode ser filtrado por essas tags para isolar o gasto real desta entrega.

O detalhamento e a evidência também estão em [ESTIMATIVA_CUSTOS_AWS.md](#).

9. Atividades Opcionais (Pontuação Extra)

Link do Perfil Público / Badge (Google Cloud Skills Boost): `[ESPAÇO PARA INSERIR URL DO PERFIL PÚBLICO]`

Anexo — Itens opcionais do enunciado já adotados

Item	Situação
Terraform organizado em módulos	7 módulos próprios + módulo de VPC da comunidade
Repositórios ECR criados via Terraform	os 5, com lifecycle policy e scan no push
Flag <code>use_lockfile</code> no backend S3	ativa nas três camadas
Roles e policies IAM via Terraform	control plane, nós, EBS CSI, CI e as duas roles de IRSA
GitHub Actions como ferramenta de CI	9 workflows
Área de GitOps em pasta do monorepo	<code>gitops/</code> , alternativa aceita pelo enunciado